

Codul sursa

Iliut Raul-Daniel G:41/2

Coman Radu-Gabriel G: 42/1

tsp_utils:

```
import random
```

```
import math
```

```
import numpy as np
```

```
SEED = 42
```

```
random.seed(SEED)
```

```
np.random.seed(SEED)
```

```
def generate_cities(n=200):
```

```
    return [(random.uniform(0, 1000), random.uniform(0, 1000)) for _ in  
            range(n)]
```

```
def distance(city1, city2):
```

```
    return math.sqrt((city1[0] - city2[0]) ** 2 + (city1[1] - city2[1]) ** 2)
```

```
def route_distance(route, cities):
```

```
    total = 0
```

```
    for i in range(len(route)):
```

```
        total += distance(cities[route[i]], cities[route[(i + 1) % len(route)]])
```

```
    return total
```

```
def random_route(n):
```

```
    route = list(range(n))
```

```
    random.shuffle(route)
```

```
    return route
```

simulated_annealing:

```
import random
import math
from tsp_utils import route_distance

def simulated_annealing(cities, initial_temp=1000, cooling_rate=0.995,
iterations=500):
    n = len(cities)

    current = list(range(n))
    random.shuffle(current)

    best = current[:]

    current_cost = route_distance(current, cities)
    best_cost = current_cost

    temp = initial_temp

    best_history = []
    current_history = []

    while temp > 1:
        for _ in range(iterations):
            new_route = current[:]
            i, j = random.sample(range(n), 2)
            new_route[i], new_route[j] = new_route[j], new_route[i]

            new_cost = route_distance(new_route, cities)

            if new_cost < current_cost or random.random() < math.exp((current_cost
- new_cost) / temp):
                current = new_route
                current_cost = new_cost
```

```

    if current_cost < best_cost:
        best = current[:]
        best_cost = current_cost

    best_history.append(best_cost)
    current_history.append(current_cost)

    temp *= cooling_rate

return best, best_cost, best_history, current_history

```

genetic_algorithm:

```

import random
from tsp_utils import route_distance

def create_population(size, n):
    population = []
    for _ in range(size):
        route = list(range(n))
        random.shuffle(route)
        population.append(route)
    return population

def tournament_selection(population, cities, k=3):
    selected = random.sample(population, k)
    selected.sort(key=lambda x: route_distance(x, cities))
    return selected[0]

def crossover(parent1, parent2):
    n = len(parent1)
    start, end = sorted(random.sample(range(n), 2))

    child = [-1] * n

```

```
child[start:end] = parent1[start:end]
```

```
ptr = 0
```

```
for city in parent2:
```

```
    if city not in child:
```

```
        while child[ptr] != -1:
```

```
            ptr += 1
```

```
            child[ptr] = city
```

```
return child
```

```
def mutate(route, mutation_rate):
```

```
    for i in range(len(route)):
```

```
        if random.random() < mutation_rate:
```

```
            j = random.randint(0, len(route) - 1)
```

```
            route[i], route[j] = route[j], route[i]
```

```
    return route
```

```
def genetic_algorithm(
```

```
    cities,
```

```
    population_size=50,
```

```
    generations=200,
```

```
    mutation_rate=0.02,
```

```
    crossover_rate=0.9,
```

```
    elitism=0.1
```

```
):
```

```
    n = len(cities)
```

```
    population = create_population(population_size, n)
```

```
    best_history = []
```

```
    current_history = []
```

```
    for gen in range(generations):
```

```
        population.sort(key=lambda x: route_distance(x, cities))
```

```

best = population[0]
best_cost = route_distance(best, cities)

best_history.append(best_cost)
current_history.append(route_distance(population[-1], cities))

if gen % 20 == 0:
    print(f'[GA] Gen {gen} Best: {best_cost:.2f}')

new_population = population[:int(elitism * population_size)]

while len(new_population) < population_size:
    parent1 = tournament_selection(population, cities)
    parent2 = tournament_selection(population, cities)

    if random.random() < crossover_rate:
        child = crossover(parent1, parent2)
    else:
        child = parent1[:]

    child = mutate(child, mutation_rate)
    new_population.append(child)

population = new_population

population.sort(key=lambda x: route_distance(x, cities))
best = population[0]
best_cost = route_distance(best, cities)

return best, best_cost, best_history, current_history

```

experiments:

```

from tsp_utils import generate_cities
from simulated_annealing import simulated_annealing
from genetic_algorithm import genetic_algorithm
import time

```

```

def run_sa_experiments():
    cities = generate_cities(200)

    cooling_rates = [0.99, 0.995, 0.999]

    results = []

    for rate in cooling_rates:
        start = time.time()

        best_route, best_cost, best_history, current_history = simulated_annealing(
            cities,
            initial_temp=1000,
            cooling_rate=rate,
            iterations=300
        )

        end = time.time()

        results.append({
            "cooling_rate": rate,
            "best_cost": best_cost,
            "time": end - start,
            "history": best_history
        })

        print(f"Cooling rate {rate} -> cost {best_cost}")

    return results

def run_ga_experiments():
    from tsp_utils import generate_cities
    import time

    cities = generate_cities(50)

```

```

mutation_rates = [0.01, 0.05, 0.1]

results = []

for rate in mutation_rates:
    print(f"\nRunning GA with mutation_rate={rate}")

    start = time.time()

    best_route, best_cost, best_history, current_history = genetic_algorithm(
        cities,
        population_size=50,
        generations=200,
        mutation_rate=rate,
        crossover_rate=0.9,
        elitism=0.1
    )

    end = time.time()

    print(f"Done. Cost = {best_cost:.2f}, Time = {end - start:.2f}s")

    results.append({
        "mutation_rate": rate,
        "best_cost": best_cost,
        "time": end - start,
        "history": best_history
    })

return results

```

main:

```
from experiments import run_sa_experiments, run_ga_experiments
import matplotlib.pyplot as plt
```

```
print("Running SA...")
sa_results = run_sa_experiments()
```

```
plt.figure()
for r in sa_results:
    plt.plot(r["history"], label=f'SA rate={r["cooling_rate"]}')"
```

```
plt.title("Simulated Annealing")
plt.legend()
plt.grid()
```

```
print("\nRunning GA...")
ga_results = run_ga_experiments()
```

```
plt.figure()
for r in ga_results:
    plt.plot(r["history"], label=f'GA mut={r["mutation_rate"]}')"
```

```
plt.title("Genetic Algorithm")
plt.legend()
plt.grid()
```

```
plt.show()
```